CS268 - Introduction to Optimization
Project Report

Berkay Guler
Ryuto Kitagawa

University of California, Irvine
December 2023

1 Problem Statement

Decentralized machine learning has gained popularity due to the prevalence of advanced machine learning algorithms in modern systems. The largest bottle neck in the machine learning field is training models to reach an acceptable level of proficiency. This motivates us to consider machine learning algorithms in the parallel settings, the focus of this project being a decentralized network of nodes. A decentralized machine learning setting is when each node in the network represents a neural network trained on local training data for some time. Once all nodes have completed a single round of training on their neural networks, a round of communication occurs where each node sends the parameters for their model to each of their neighbors, and the node computes a weighted average to decide the node's new aggregated result. This process is repeated as many times as necessary until some stopping criteria is met. In the current field of research, the most focus is placed on an uneven distribution of the data, where some of the nodes have a share of the training data than others.

More formally, given a set of distributed nodes $\mathcal{N} = \{1, 2, \ldots, n\}$, $\mathcal{D}_i$ denotes the dataset of the ith node where $\mathcal{D} = \bigcup_{i=1}^{n} \mathcal{D}_i$ is the total dataset available and $\mathcal{D}_i \cap \mathcal{D}_j = \emptyset$ for $i, j \in \mathcal{N}$ where $i \neq j$. Each node trains a local multiclass classificaton model $h_{\theta_i} : x \in \mathbb{R}^d \to y \in \mathbb{R}^c$ parameterized by θ_i.

We denote each local dataset as a set of feature-label pairs

$$\mathcal{D}_i = \{(x_1^{(i)}, y_1^{(i)}), (x_2^{(i)}, y_2^{(i)}), ..., (x_{k_i}^{(i)}, y_{k_i}^{(i)})\}$$

where $x_i \in \mathbb{R}^d$ is the feature vector, $y_i \in \mathbb{R}^c$ is the label, k_i is the number of data points at node i, and c is the number of classes.

The topology of the nodes are represented by a weighted directed graph $G(V, E)$ with set of nodes V and edges E. The topology has an adjacency matrix W

with entries $W_{ij} = 0$ if two nodes are disconnected and $W_{ij} \in \mathbb{R}^+$ if the nodes i and j are connected with weight W_{ij}.

We minimize the cross entropy loss L to obtain θ_i^* for each model h_{θ_i}.

$$\theta_i^* = \underset{\theta_i}{\operatorname{argmin}}\, L(\mathcal{D}_i, h_{\theta_i})$$

Considering the above setup, we employ the decentralized stochastic gradient descent (algorithm 1) to train local models.

Algorithm 1: Decentralized Stochastic Gradient Descent

Data: Initial parameters $\theta_i(0)$ for each node i, step size α, maximum iterations T, and weight matrix W
Result: Optimized parameters θ_i for each node i
for $t = 0$ **to** $T - 1$ **do**
  for *each node i in parallel* **do**
    Sample $Z_i = \{(x_j^{(i)}, y_j^{(i)}) | j = 1, 2, ..., N\}$ from $\mathcal{D}_i$;
    Compute stochastic gradient: $\nabla_{\theta_i} L(Z_i, h_{\theta_i})$;
    Collect weights from neighbours: $\theta_i(t + 1/2) = \sum_{j \in \mathcal{N}_i} W_{ij}\theta_j(t)$;
    Update weights with gradient:
    $\theta_i(t + 1) \leftarrow \theta_i(t + 1/2) - \alpha \nabla_{\theta_i} L(Z_i, h_{\theta_i})$;

The performance of algorithm 1 is highly dependent on the topology. As the connectivity of the topology increases the convergence rate of the algorithm improves with increasing communication costs. Our aim is to optimize the average accuracy across nodes subject to constraints on the maximum degree of graph and number of nodes by utilizing the trade-off between the connectivity and communication overhead. Therefore, the optimization objective looks like the following:

$$\max_G \frac{1}{n} \sum_{i=1}^{n} P_G(h_{\theta_i^*}, \mathcal{D}_i)$$

$$\text{subject to } \max_{i \in V}(deg(i)) \leq k$$

where P_G is the accuracy obtained after decentralized training with topology G and $deg(i)$ denotes thew maximum of in degree and out degree of node i.

2 Work Decomposition

2.1 Berkay Guler

The problem described in section 2 cannot be solved directly because the analytical form of the function P_G is not known and sampling from this function is expensive in time and computation especially for large datasets and networks. Therefore, I approached the problem empirically and tested various graph topologies with different connectivity constraints and trained decentralized networks for larger number of nodes and dataset. I implemented the decentralized stochastic descent optimization algorithm and helper modules in Python using PyTorch, and Networkx libraries.

2.2 Ryuto Kitagawa

For my part of the project, I implemented 3 different optimization algorithms: the Generalized Pattern Search, Nelder Mead, and a combination of these two solutions to create a third possible solution. I had written the code to test each of these algorithms as well as analyzing their results. In summary, I was responsible for the black box algorithms section of this project.

3 Experience Solving and Coding the Problem

While the optimization equation is simple to write out, using traditional optimization techniques on the equation is difficult. Due to the underlying equation for the accuracy of a given network and the constraint on the out degree are both non-differentiable, we are left to only consider black-box styles of optimization algorithms and empirical analysis.

The other major issue with this problem is the amount of processing time required to evaluate each point. Many optimization algorithms often use hundreds of thousands of evaluations to the objective function P. However since each evaluation may take minutes to compute, making hundreds of thousands of calls to P would make the problem infeasible to solve.

Considering the two factors above, two natural approaches rose. One approach would be to empirically test various network topologies to see what works best for the problem at hand characterized by the number of nodes, dataset, and connectivity. The second approach would be to exclusively utilize black box optimization algorithms with a minimum number of objective function calls necessary.

3.1 Black Box Algorithms

Some black box optimization algorithms could easily be ruled out due to the nature of them utilizing a line search algorithm, which often requires bracketing

and calls the objective function an unknown number of times. These algorithms include cyclic coordinate descent and Powell's method.

Hooke-Jeeves initially appeared to meet the criteria for a black box algorithm which solves the optimization algorithm in as few function calls as possible. However, the issue with this approach lies in the nature of the problem. Hooke-Jeeves arbitrarily either increases or decreases each parameter of the input by some learning step to search the space of possible solutions. In the case where we are considering an upper bound on the out degree, the algorithm would only ever consider increasing or decreasing the weight of an edge independently. Since we are considering a weighted average of the output, the optimization algorithm should be increasing one edge and decreasing in another in order to ensure the sum of the weights is still 1. This simultaneous consideration of both options is a crucial aspect of the problem, which the Hooke-Jeeves method does not cover.

Conveniently, there is a similar approach but has the necessary flexibility for this problem, the Generalized Pattern Search algorithm. This algorithm is similar to Hooke-Jeeves, the only difference being which directions that are searched can be manipulated by the user (along with a few additional optimizations). The directions provided were for each edge that is increased in one edge, must have taken it's value from a different edge with the same end node. This ensures the sum of weights on any in degree is 1. The code for the optimization algorithm itself was adapted from [KW19].

Another useful approach to this problem is the Nelder-Mead method. This approach to the problem is particularly nice as several iterations which affect the stopping criteria in the algorithm only requires a constant number of evaluations to the objective function. This property means so long as the shrinking step is not called too often, then the amount of time it would take to solve the problem would be negligible. The code here was also adapted from [KW19].

3.2 Combination Solution

The previous section discussed solutions utilizing only a single optimization algorithm. The two optimization algorithms which seemed best suited for the task were Generalized Pattern Search and the Nelder-Mead method. While both of these approaches have their advantages, each of these optimization algorithms may be able to solve different aspects of the problem when we are considering a limited number of graphs.

For the case where the out degree of each node is constrained, the Generalized Pattern Search algorithm, with the directions provided, are unable to change how the nodes are connected to each other unless the learning rate is exactly the same as the value of the edge weight. While the additional directions could be added to the code, this would significantly increase the number of directions

in which the algorithm would have to search per iteration which would make searching the space of possible solutions take much longer.

The issue with the Nelder Mead method when restricting the number of degrees is the Nelder Mead algorithm typically increases or decreases all of the parameters simultaneously. Since the search space is not convex, this causes many of the points the Nelder Mead method attempts to generate are invalid, with no obvious way to push the algorithm to a valid solution, due to the discrete nature of the problem. This is a severe issue in the case of restricting the out degree, but the issue is less prominent in a fully connected graph.

Therefore, a combined approach may use the flexibility of the Generalized Pattern Search to first find how nodes should be connected to each other as the first step of the optimization. Then the Nelder Mead method may then find what the set of weights should be. This combined approach should theoretically have the same output as simply using the Nelder Mead optimization algorithm on a fully connected graph, and thus due to time constraints, will only be tested on the restricted out degree setting.

3.3 Empirical Approach

When the number of nodes and the dataset size pose great computational challenges, use of black box algorithms becomes impractical. Considering that the optimal topology for a given decentralized learning problem is dependent upon various factors like the dataset, learning problem, communication budget, and computation budget, it is desirable to empirically analyze the role of topology on the average accuracy for the problem at hand [Pal+23]. Following the practices adopted in the literature on the subject [Kav+21], we try and test the fully connected topology and four other topologies on the decentralized learning of digit recognition problem with 100 nodes and different connectivity levels. We provide an empirical analysis to find a good trade-off between connectivity and convergence by interpreting the results of various simulations.

3.4 Coding Challenges

3.4.1 Black Box Algorithms

One of the most challenging aspects of implementing a black box approach for algorithms was managing the time each trial consumed. Given that each iteration of the algorithm required several minutes and optimization algorithms typically make thousands of calls to the objective function f, a single function evaluation taking even a minute was impractical. Our initial strategy was to parallelize the computations to enhance speed. Although this may have been beneficial, the process of parallelization itself was time consuming and it was uncertain whether it would significantly improve performance to justify the effort. Consequently, we chose to address this issue by substantially reducing

the problem size for the black box algorithms. This adjustment ensured that each function evaluation only took a few seconds, rather than minutes.

3.4.2 Empirical Approach

Since the decentralized learning task is simulated on one machine rather than on distributed computing resources, we could not use datasets and models with high dimensions, which might not be the case in a real life application. Instead we limited our experiments to MNIST dataset [LC10] which consists of 28x28 grayscale images, and a neural network with 2 fully connected layers with 512 neurons and around 0.6 million learnable parameters in total.

4 Results

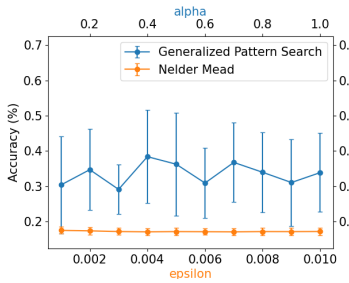
4.1 Fully Connected Decentralized Experiments with Black Box Algorithms

We start by examining the fully connected setting, which offers insights into the performance of both the Generalized Pattern Search and Nelder Mead algorithms. This setting helps identify more optimal parameters in a constrained setting. Our test network comprises 3 nodes which are fully connected. 1 node contains 640 points of training data and the others containing 64 data points each. The evaluation of the whole test network uses 640 data points not contained by any of the other nodes. Despite the simplicity of this setup, running about 20 different parameter configurations took over 15 minutes, demonstrating the need for such a minimal setting.

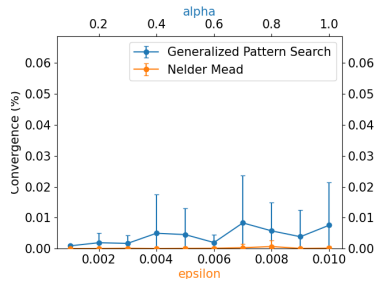
A key observation from our experiments, which will be evident in the subsequent graphs, is the high variability in the results. Part of this variance is inherent to the problem, due to the many local minima the optimization algorithm may converge to, depending on the starting conditions. Ideally, this variance could be mitigated through a sufficient number of trials, but this was not feasible for this project due to the time constraints mentioned earlier.

In Fig 1, we compare the performance of the Generalized Pattern Search and Nelder Mead optimization algorithms under the aforementioned conditions. For the Generalized Pattern Search, γ was fixed to be 0.5, varied α between 0.1 and 1 and set $\varepsilon = \alpha \cdot \gamma^5$, ensuring the learning rate is halved five times regardless of the α value. For the Nelder Mead algorithm, ε ranged between 0.001 and 0.01 while the other parameters were fixed to $\alpha = 1.0$, $\beta = 2.0$, and $\gamma = 0.5$.

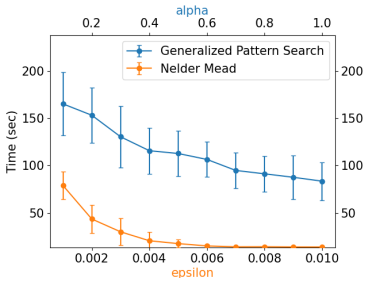
In Fig 5a, we observe on average the Generalized Pattern Search method outperforms the Nelder Mead algorithm for the accuracy of the weight matrix. However, due to high variance and a limited number of trials, it is challenging



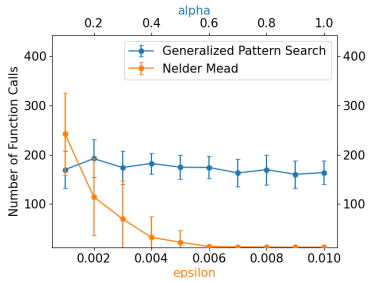
(a) Accuracy on test data measured by percentage



(b) Convergence of accuracy on test data



(c) Time to run optimization algorithm in seconds



(d) Number of calls to objective function each function call makes

Figure 1: Comparing the performance different optimization algorithms on 3 nodes, fully connected, with 10 rounds of communication, training data unfairly distributed across 25 trials

to tell if the performance improves as α increases. Though performance does seem consistent across different values of α.

Interestingly, the Nelder Mead algorithm maintains an average performance of 0.175 across varying ϵ values. As a baseline, we compared this to the weight matrix with the best performance across 9 random adjacency matrices, where the Nelder Mead algorithm performed slightly better with the random method obtaining an average of 0.171. However, the limited number of trials constrains the robustness of this conclusion.

One notable observation is the large standard deviation in the Generalized Pattern Search results, contrasted with Nelder Mead optimization results where the error bars are not visible at all. This suggests the Nelder Mead method may be less suitable for this problem, possibly due to the method of selecting initial matrices for the starting simplex. An alternative approach for simplex selection was considered but deemed impractical due to constraints in the objective function's implementation. The objective function uses a square error to punish any solution outside of the constraints of the problem, but since the considered

simplex would have resulted in the exact same output for the objective function, the Δ in the simplex method would have been 0, regardless of the output.

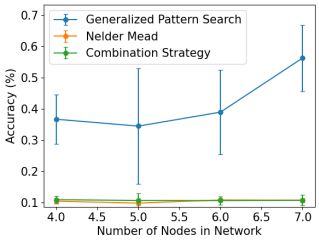
In Fig 5b, we see that the convergence rate of the Generalized Pattern Search Algorithm increases with the learning rate. This implies that a lower ϵ combined with a higher learning rate could yield better solutions. However, the Nelder Mead method showed zero convergence rate across all ϵ values. This convergence rate provides us less information as the Nelder Mead method actively keeps track of a list of points of the simplex, thus the optimal x found so far may remain the same for several iterations as the rest of the values are modified to lower the Δ value.

Figures 5c and 5d focus on time performance, measured by wall time and the number of calls to the objective function respectively. The Generalized Pattern Search algorithm, despite its significantly superior output, does also take considerably longer than the Nelder Mead method. Intriguingly, for the smallest values of α and ϵ, the Nelder Mead algorithm makes more function calls than the Generalized Pattern Search algorithm but still operates faster. The reason for this is unclear, though some background python optimization is suspected to be the culprit. Both algorithms show decreased wall time with increasing parameter values. These results have the least variance, lending them higher reliability, though the limited trials still warrants a cautious interpretation of these results.

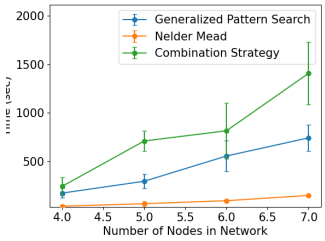
Overall, the data suggests the Generalized Pattern Search algorithm outputs a similar result for any α. Therefore, the α with the most time efficiency, 1, is the most optimal. The Nelder Mead algorithm, while significantly faster, offers a marginal improvement over taking the best out of $n + 1$ randomly generated adjacency matrices. Since a larger ϵ did not provide a notable improvement to result of the optimization algorithm and only a marginal detriment in time cost, 0.001 was chosen, prioritizing iterations and output performance over speed. Unfortunately, the prospects for the Combination algorithm in the restricted out degree setting are not particularly promising

4.2 Restricted Degree Decentralized Experiments with Black Box Algorithms

Here we use the Generalized Pattern Search, Nelder Mead, and Combination algorithm to see how these algorithms perform in the restricted degree setting. Using the results from the previous section, we have chosen the parameters for the optimization algorithms accordingly. We vary the number of nodes between 4 through 7, with a max degree of 3, and 10 rounds of communication. The distribution of the data was done in a similar fashion to the previous section for the same reason as was listed.
Fig 2 presents a comparison of the three algorithms. It is clear the Generalized Pattern Search algorithm outperforms the other two methods substantially. Re-



(a) Accuracy on test data measured by percentage



(b) Time to run optimization algorithm in seconds

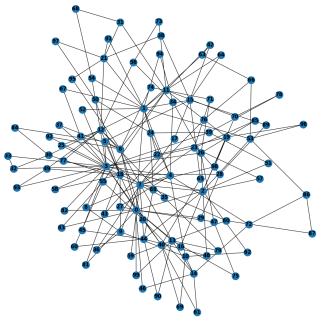
Figure 2: Comparing the performance of different optimization algorithms, with maximum out degree of 3, with 10 rounds of communication, training data unfairly distributed, across 10 trials

grettably, the Combination algorithm, which was intended as an enhancement, not only requires more time to execute than the individual optimization algorithms but also yields results comparable to the original Nelder Mead algorithm. Based on the results previously seen of how well the Nelder Mead algorithm performed in both experiments, it is evident this is the limiting factor in the optimization algorithm. A more effective approach may have been to integrate the Generalized Pattern Search in place of the Nelder Mead algorithm, resulting in a two stage Generalized Pattern Search algorithm. Another possibility would be to adopt the strategy suggested in Sec 3.2, which involved offering all possible directions to the Generalized Pattern Search algorithm, even with the time cost.

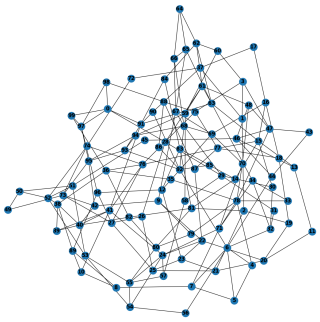
4.3 Empirical Analysis on Larger Networks and Dataset

Motivated by the impracticality of directly optimizing the objective function, we present experimental results on various settings to investigate the effect of the connectivity characteristics of the network on the performance of the decentralized SGD algorithm. Experiments are performed with 100 node, 0.001 learning rate, i.i.d. data across nodes, and 2000 rounds communication budget. As outlined in algorithm 1, we sample 32 data points and perform three updates locally before each communication round. We carry out our experiments for complete graph, Albert Barabasi Graph, Erdos-Renyi Random Graph, regular random graph, and Watts-Strogatz small world graph. For sparse graphs, we repeat experiments with degrees 2, 4, 6, 8, 10, 20 and 30.

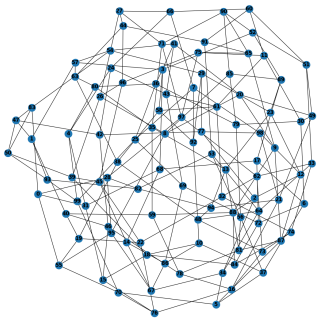
We evaluate the performance of the decentralized SGD under different topologies on figure 4. We observe that the complete graph lags behind all other sparse graphs initially yet eventually outperforms them. The same phenomenon is also valid for random regular graph. We attribute this behaviour to homogeneous connectivity of these two graphs. Another observation about connectivity is



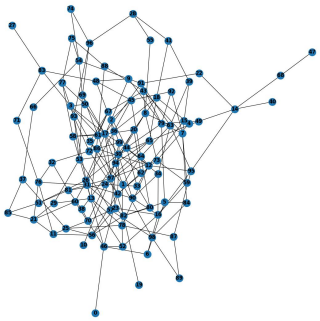
(a) Barabasi Albert Graph Model



(b) Watts–Strogatz Small World Graph Model

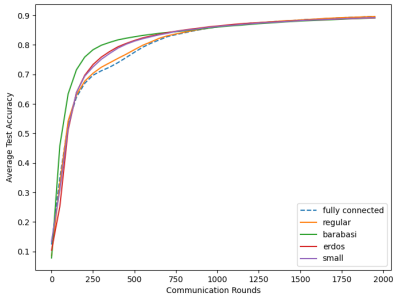


(c) Random Regular Graph

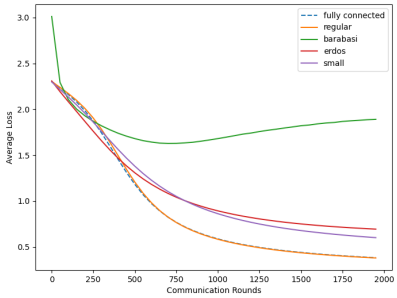


(d) Erdos-Renyi Random Graph Model

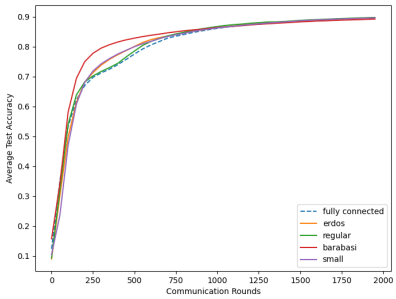
Figure 3: Examples of Sparse Graphs Used for Experiments



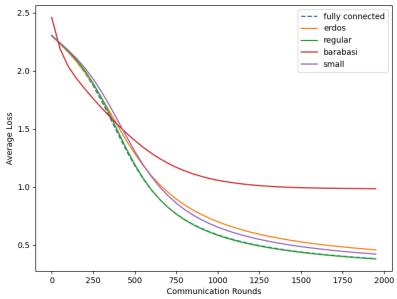
(a) Average Accuracy on Degree 4 Sparse Graphs



(b) Average Loss on Degree 4 Sparse Graphs



(c) Average Accuracy on Degree 20 Sparse Graphs



(d) Average Accuracy on Degree 20 Sparse Graphs

Figure 4: Comparison of Topologies on Convergence

that the performance gap between topologies becomes less pronounced as the sparsity of the graphs decrease. Among all topologies, we see that the Barabasi Albert graph model [BA99] performs the poorest as the communication budget increases although it outperforms all other topologies for communication budgets below 500 rounds. Therefore, we conclude that for iid data, homogeneous topologies outperform others if the number of communication budget is high enough. On the other hand, heteregeneous connectivity helps the average accuracy to increase rapidly in earlier rounds.

As shown in figures 5a and 5b, Barabasi-Albert graph leads to a failure to converge to a good solution even for unrealistic connectivity levels such as with degree 20 and 30. Moreover, some of the nodes diverges with degree 2 setting, leading to failure of some of the nodes learning a meaningful model and disturbing the neighbouring nodes' models with their poor model.

Random regular graph topologies turn out the be the most stable with respect

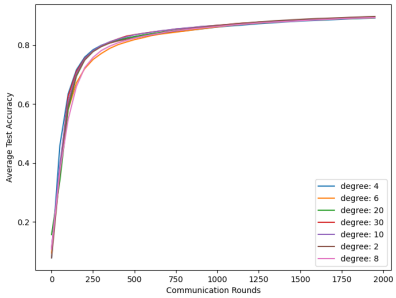
to sparsity as seen in figures 5c and 5d. However, random regular graphs are not realistic graphs since in general nodes tend to have different degrees in real life networks. We also observe that increasing the connectivity of a random regular graph provides a minor marginal benefit but increases the communication costs greatly. Therefore, we conclude that random regular graph topology can be used with degrees as low as 2 to result in a decent convergence.

Finally, we analyze the Watts-Strogatz small world **??** graph in figures 5e and 5f and conclude that it is the most powerful candidate in our experimenrts in that it does not suffer from huge performance drops for low connectivity settings and stands out as being less artificial compared to the random regular graphs.

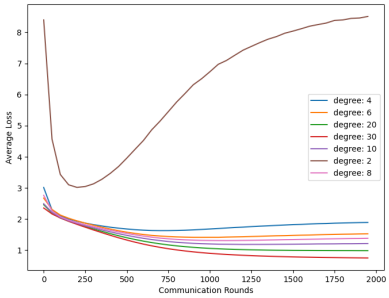
5 Conclusion

This project evaluated the efficacy of optimization algorithms, specifically the Generalized Pattern Search, Nelder Mead, and a novel Combination algorithm, for solving a decentralized machine learning problem. The Generalized Pattern Search algorithm outperformed the Nelder Mead algorithm in the output, which was able to achieve a higher accuracy. As for the Nelder Mead algorithm, while faster, often resulted in substantially sub-optimal outcomes. This disparity in performance was particularly evident in the restricted out degree setting, where both the Nelder Mead and Combination algorithms performed substantially worse than the Generalized Pattern Search algorithm. As for the analysis for large networks and datasets, the Watts-Strogatz model has been found to provide a good trade-off for the communication overhead and convergence rate among all the topologies considered, whereas random regular graph performing the best in every aspect even though being too artificial to arise in real-life scenarious.

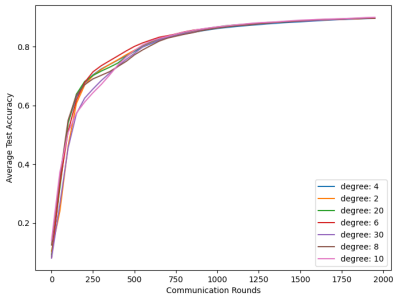
There are several areas of this project where, upon further consideration, could have been vastly improved upon. For the Nelder Mead algorithm, the values for α and β should have been experimented with more, as those values were arbitrarily fixed based on the values found in [KW19]. However more critically, it should have been noted that the Nelder Mead algorithm had the same flaw in its optimization algorithm as the Hooke-Jeeve's method as well. Therefore, different iterations of the Generalized Pattern Search Algorithm could have been considered. A recent paper [VHJ22] had also suggested looking into the possibility of optimizing for a heuristic which would not have required any function calls to our original objective function, and instead optimizing for the spectral gap, which has been found in research to also provide good graphs. This would have been a good avenue to look for when considering the black box optimization algorithms.



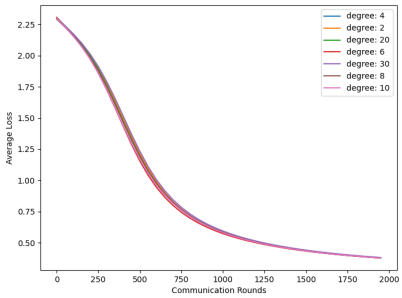
(a) Effect of Degree on Accuracy for Barabasi Albert Model



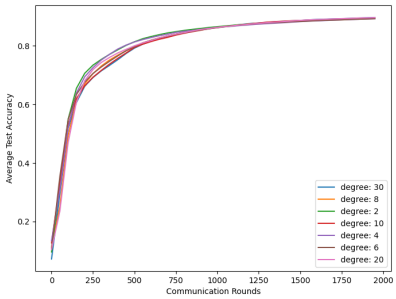
(b) Effect of Degree on Loss for Barabasi Albert Model



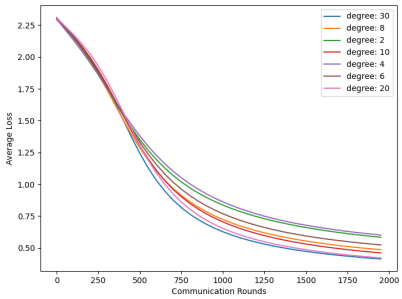
(c) Effect of Degree on Accuracy for Random Regular Graph



(d) Effect of Degree on Loss for Random Regular Graph



(e) Effect of Degree on Accuracy for Watts-Strogatz Model



(f) Effect of Degree on Loss for Watts-Strogatz Model

Figure 5: Effect of Graph Degree on Convergence

References

[BA99] Albert-Laszlo Barabasi and Reka Albert. "Emergence of Scaling in Random Networks". In: *Science* 286.5439 (1999), pp. 509–512. DOI: 10.1126/science.286.5439.509. eprint: http://www.sciencemag.org/cgi/reprint/286/5439/509.pdf. URL: http://www.sciencemag.org/cgi/content/abstract/286/5439/509.

[LC10] Yann LeCun and Corinna Cortes. "MNIST handwritten digit database". In: (2010). URL: http://yann.lecun.com/exdb/mnist/.

[KW19] Mykel J. Kochenderfer and Tim A. Wheeler. *Algorithms for Optimization*. The MIT Press, 2019. ISBN: 0262039427.

[Kav+21] Hanna Kavalionak et al. "Impact of Network Topology on the Convergence of Decentralized Federated Learning Systems". In: *2021 IEEE Symposium on Computers and Communications (ISCC)*. 2021, pp. 1–6. DOI: 10.1109/ISCC53001.2021.9631460.

[VHJ22] Thijs Vogels, Hadrien Hendrikx, and Martin Jaggi. *Beyond spectral gap: The role of the topology in decentralized learning*. 2022. arXiv: 2206.03093 [cs.LG].

[Pal+23] Luigi Palmieri et al. "The Effect of Network Topologies on Fully Decentralized Learning: A Preliminary Investigation". In: *Proceedings of the 1st International Workshop on Networked AI Systems*. NetAISys '23. Helsinki, Finland: Association for Computing Machinery, 2023. ISBN: 9798400702129. DOI: 10.1145/3597062.3597280. URL: https://doi.org/10.1145/3597062.3597280.